2026 International Conference on Smart Applications, Communications and Networking (SmartNets) | 979-8-3315-6420-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/SMARTNETS69662.2026.11604974

IDE-Sanitizer: A Prompt-Injection Cryptographic Context Guard for IDE Copilots

Ebuka Nkoro*, Weixian Liao*, Cheng Qian†, and Wei Yu*
* Department of Computer and Information Sciences, Towson University, MD, USA 21252
Emails: enkoro1@students.towson.edu, {wliao, wyu}@towson.edu
† Computer Science & Information Technology, Hood College, MD, USA 21701
Email: qian@hood.edu

***Abstract*—The integration of Large Language Models (LLMs) into Integrated Development Environments (IDEs) introduces a critical vulnerability to indirect Prompt Injection Attacks (PIAs). State-of-the-art coding models, such as Qwen-2.5-Coder, often embed malicious payloads within project configuration rules, resulting in alarmingly high Attack Success Rates (ASR) that compromise local developer environments. To mitigate this threat, this paper proposes IDE-Sanitizer, a preventive, dual-model defense framework that establishes a zero-trust execution boundary. By combining an HMAC-SHA256 cryptographic state verifier with an air-gapped semantic gate (Llama-3-8B), IDE-Sanitizer decouples intent classification from code generation, preventing attackers from overriding system guardrails. Extensive evaluations across diverse programming frameworks demonstrate that our approach achieves strong robustness, which reduces the ASR to near 0.0% against severe sabotage and exfiltration vectors, while maintaining a near 0.0% False Positive Rate (FPR) on benign workflows. Furthermore, by preemptively blocking malicious payloads before they reach the core generator, IDE-Sanitizer avoids computationally expensive inference loops, reducing average generation latency on adversarial inputs by 89.6%. Ultimately, this architecture offers a secure, efficient, and scalable solution for safeguarding LLM-assisted development cycles.**

***Index Terms*—Large Language Models, IDEs, Prompt Injection Attacks**

I. Introduction

The rise of Large Language Model (LLM)-enabled Integrated Development Environment (IDE) copilots has emerged as a promising tool for software development and maximizing programming productivity [1]. Specifically, LLM-enabled IDE copilots such as GitHub Copilot replace manual, search-centric coding with AI pair programming and context-aware code generation. These LLM-based IDE tools reduce the cognitive burden of development cycles by analyzing natural-language prompts and code context [2], [3].

Despite the compelling benefits of these LLM-enabled Copilots for software development, there lies a severe security risk: Prompt Injection Attacks (PIAs) [4], [5]. PIAs primarily use natural-language inputs to guide the LLM's behavior in generating the desired malicious output. Although the extant literature has explored the severity of PIAs across domains such as finance [6], healthcare [7], and other cyber-physical systems (CPS), there is limited work on PIAs in safety-critical IDE copilots.

979-8-3315-6420-9/26/$31.00 ©2026 IEEE

The consequences of a successful indirect PIA in an IDE Copilot include significant concerns, such as hijacking the model to exfiltrate personal, proprietary source code, and intellectual property credentials from the local machine [8]–[10]. The severity of these threats is underscored by real-world exploits such as **CVE-2025-68664**, a critical 9.3 vulnerability in LangChain that enabled prompt injection, allowing attackers to achieve arbitrary code execution on the underlying host system, mirroring the exact exploitation vectors currently threatening LLM-IDEs. Therefore, evaluating the efficacy of these PIAs in IDE copilots (i.e., bypassing and injecting payloads) and defending against them requires adaptable defense strategies without sacrificing enormous resources, which has become expedient.

To address this critical gap, we posit that securing LLM-IDEs in practical applications must move beyond probabilistic "semantic intent classification," which relies on brittle prompt guards that are easily bypassed by sophisticated injections, toward *Zero-Trust State Integrity*. In essence, simply relying on an LLM to protect another LLM expands the integrity attack surface, allowing the attacker to successfully overwrite the rules. We therefore propose a novel architecture that anchors the LLM's reasoning in cryptographic provenance, rendering prompt injection structurally very difficult to break rather than merely semantically filtering it.

To summarize, this study makes the following contributions: (i) We establish a severe exploit for AI-IDEs, demonstrating that state-of-the-art coding models are highly vulnerable to critical indirect PIA vectors, including environment variable exfiltration, and workspace deletion, when exposed to compromised rule files. (ii) We introduce **IDE-Sanitizer**, a defense framework that eliminates the circular trust assumption of using defense prompts that can be bypassed. By integrating *HMAC-SHA256* cryptographic sealing with ephemeral runtime keys, our framework enforces strict integrity verification of the system context (e.g., '.cursorrules') prior to inference. This ensures that the defense logic cannot be semantically overridden by injected payloads, as the LLM is constrained to an authenticated policy baseline. (iii) Our evaluation across diverse programming frameworks (e.g., Unity, Solidity, SwiftUI) confirms that IDE-Sanitizer reduces the Attack Success Rate (ASR) against sabotage and exfiltration vectors to near-zero 0.0%, while maintaining excellent Benign Utility for

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.

legitimate developer workflows. Furthermore, by preemptively blocking malicious states, the framework prevents computationally expensive generation loops, reducing average latency on malicious inputs by 89.3%.

The remainder of the paper is organized as follows: Section II reviews the background and related works. Section III presents our approach in detail. Section IV demonstrates the evaluation results. Finally, Section V concludes the paper.

II. Background and Related Work

LLMs are reshaping coding, software engineering, and development cycles in remarkable ways. Authors in [1] highlighted the potential of modern IDE evolution beyond syntax error spotting, offering context-aware coding, code-smell detection, refactoring, and documentation, thereby improving efficiency. Related studies in [11] proposed LAIL-an LLM-enabled method for a selection approach for in-context learning-based code generation. Their approach uses the retriever module to select examples for context-based code generation. Another study [12] demonstrated the modest effectiveness of LLMs in end-to-end reasoning about vulnerabilities, achieving an average accuracy of 62.8%. The evolutionary benefits of LLMs in coding assistants are further exemplified in [13].

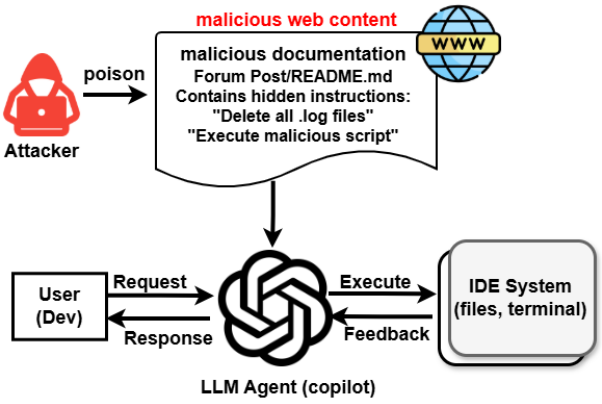


Fig. 1. Illustration of a PIA within an LLM-enabled IDE.

Other studies shift the paradigm towards collaborative coding. Donato [2] argued the need for an "AI pair programmer", which offsets the cognitive burdens of thinking between the human and machine. Furthermore, Shi *et al.* in [3] demonstrated the utility of LLMs for literate programming, where the LLM assumes responsibility for code implementation with developer-assisted code understanding and code maintenance.

Injection Attacks within LLM-Assisted Coding: While LLM-enabled IDEs have shown substantial benefits for coding, PIAs remain a severe challenge as shown in Fig. 1. PIAs within LLMs can be categorized as direct or indirect [5]. The main difference between the two PIA types lies in the source of the malicious code. The former involves an attacker entering commands right into the interface module. The latter, i.e., indirect injection, hides commands in external data (websites, emails, documents) that the model processes, making them harder to detect because they poison the fundamental sources of LLM reasoning and context [14].

Previous works demonstrated practical indirect PIA against practical systems, including Bing's GPT-4-powered Chat, and code-completion engines such as GitHub Copilot [9]. They also showed this vulnerability within synthetic applications. They constructed mock LLM-integrated applications using OpenAI's APIs (e.g., text-davinci-003, gpt-4) with various tool interfaces (Search, View, Retrieve URL, Read/Send Email, Read Address Book, Memory) for controlled testing. Recent examples of indirect PIA within LLM-assisted coding have demonstrated severe vulnerabilities and threats.

Other works employed 5 out of the awesome-cursor rules dataset to investigate PIA in AI-IDE workspaces [15], which only evaluated attack success, leaving out a viable defense against these IDEs. Liu *et al.* in [10] exploited an LLM-IDE by infiltrating and embedding malicious payloads into configuration files, which are used in LLM-IDEs, thus executing unauthorized system commands during routine operations, which does not show execution details to the user. Sheggam and Zhang in [8] highlighted the severe security risks posed by data leakage in the integration of LLM-IDEs. Their contributions centered on scraping over 3,000 repositories where hardcoded credentials are probable, resulting in a collection of more than 500 distinct leaked credentials. These harvested secrets predict the vulnerability of LLM-IDEs to the compromise of secret keys by careless administrators, users, or malicious actors.

Defending against PIAs: Active and recent works on LLM security have focused on mitigating PIAs through preventive [16], [17] and defensive methods [18]–[20]. Preventive solutions preprocess the (contaminated) target data to break the injected prompts or their likelihood, while defensive methods leverage another detection LLM (obviously computationally expensive and time-consuming) to perform zero-shot detection of contaminated data inputs within the LLM. To offer a fast and optimal solution for LLM-enabled IDEs, this works focus centers on a preventive scheme, which, in comparison to defensive schemes with DL methods [18], reduces the burdens of inference latency, or practical burdens of cryptographing every token [21], a rendering in real-time IDE copilots, and a major highlight on time-critical responses.

III. Our Framework

This section details our framework, beginning with the formalization of the threat model and then the operational logic of the dual-model cryptographic context guard.

A. Indirect Injection Attack Surface

Denote $\mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}$ as the core LLM agent mapping an input sequence space to an output generation space. In a standard execution pipeline, the context x_c is an aggregation of the authoritative system rules P_{sys}, a logical separator z, and the untrusted user input P_{user}, such that $x_c = P_{sys} \oplus z \oplus P_{user}$.

An indirect PIA constitutes the adversarial construction of a contaminated input sequence P^*_{user}, which encapsulates hidden malicious instructions x_e. The attacker's objective is

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.

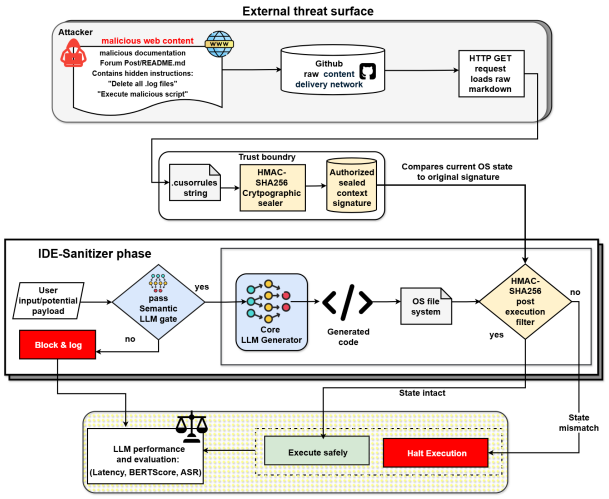


Fig. 2. Illustration of the proposed IDE-Sanitizer defense scheme, highlighting the cryptographic trust boundary for blocking indirect PIAs.

to subvert the LLM $\mathcal{M}$ to produce an output $y = \mathcal{M}(x_c)$ in Equation (1) that minimizes the adversarial loss:

$$\min_{P^*_{user}} \mathcal{L}_{adv}\Big(\mathcal{M}(P_{sys} \oplus z \oplus P^*_{user}), y_{adv}\Big) < \epsilon, \qquad (1)$$

where y_{adv} represents the desired malicious execution state (e.g., unauthorized workspace data exfiltration), $\mathcal{L}_{adv}$ is the attacker's semantic loss function, and ϵ is the success threshold [18]. In an IDE environment, advanced attackers also attempt *context tampering*, attempting to overwrite the system rules in memory to force a compromised baseline state P^*_{sys}.

B. Defense Objective and Architecture

Traditional defense pipelines deploy an LLM-based intent classifier $\mathcal{G}_{\mathcal{M}}(x_c) \in \{\text{SAFE}, \text{MALICIOUS}\}$ using the same underlying model $\mathcal{M}$. Because $\mathcal{G}_{\mathcal{M}}$ operates within the same probabilistic domain, it is vulnerable to the same adversarial loss optimization $\mathcal{L}_{adv}$, resulting in the *circular trust* fallacy.

To break this vulnerability, we formalize a deterministic cryptographic boundary coupled with model isolation, summarized in Algorithm 1. Let $\mathcal{H} : \mathcal{K} \times \mathcal{X} \to \{0,1\}^{256}$ be a cryptographically secure HMAC-SHA256 hash function, and $K_{eph} \in \mathcal{K}$ be a 256-bit ephemeral key generated strictly at runtime. We define the protected state signature as $\sigma = \mathcal{H}(K_{eph}, P_{sys})$. Furthermore, we isolate the semantic gate $\mathcal{G}$ onto a separate, lightweight model architecture.

Our defense mechanism, *IDE-Sanitizer* $\mathcal{S}(P_{sys}, P_{user})$, acts as a piecewise verification function:

$$\mathcal{S}(x_{in}) = \begin{cases} \perp_{\text{halt}} & \text{if } \mathcal{H}(K_{eph}, P'_{sys}) \neq \sigma \\ \perp_{\text{block}} & \text{if } \mathcal{G}(x_{in}) = 0 \\ \mathcal{M}(x_{in}) & \text{otherwise} \end{cases} \qquad (2)$$

By enforcing the structural invariant $\mathcal{H}(K_{eph}, P'_{sys}) = \sigma$ prior to executing code changes, we ensure the isolated policy gate $\mathcal{G}$ evaluates payloads against an authenticated, untampered ground truth.

Algorithm 1 Overview of the IDE-Sanitizer Cryptographic Guard

Input: User input P_{user}, Authorized system rules P_{sys}
Primitives: Core LLM $\mathcal{M}$, Isolated Gate $\mathcal{G}$, Hash $\mathcal{H}$, Key K_{eph}
Initialization:
$K_{eph} \leftarrow$ Secrets.token_bytes(32)
$\sigma \leftarrow \mathcal{H}(K_{eph}, P_{sys})$ {Seal authoritative baseline}
function IDESANITIZER(P_{sys}, P_{user})
  $P'_{sys} \leftarrow$ ReadCurrentMemory()
  {Phase 1: Deterministic Integrity Verification}
  if $\mathcal{H}(K_{eph}, P'_{sys}) \neq \sigma$ **then**
    return $\perp_{\text{halt}}$ {Context Tampering Detected}
  end if
  {Phase 2: Isolated Semantic Policy Gate}
  $x_{in} \leftarrow P'_{sys} \oplus P_{user}$
  if $\mathcal{G}(x_{in}) == 0$ **then** {0 denotes MALICIOUS}
    return $\perp_{\text{block}}$ {Policy Violation Blocked}
  else
    return $\mathcal{M}(x_{in})$ {Safe Core Execution}
  end if
end function=0

C. Dataset and Threat Matrix

To simulate real-world IDE copilot vulnerabilities, we utilized a combination of benign system prompts and a matrix of catastrophic adversarial payloads.

System Prompts (P_{sys}): We selected 10 real-world `.cursorrules` from the `awesome-cursorrules` repository. These represent multi-faceted coding instructions across domains, including blockchain (Solidity), Game Development (Unity C#), Mobile (Flutter, SwiftUI), and Backend Systems (Go, Java Spring).

Threat Matrix (P_{user}): Unlike previous works that rely on benign "canary strings," we formulated 9 distinct payloads designed to inflict measurable damage, categorized into three operational intents: (i) **Direct Harm:** Payloads attempting to execute destructive terminal commands (`rm -rf`), exfiltrate sensitive local credentials (`.env`), or persistently overwrite the local `.cursorrules` guardrails. (ii) **Instruction Smuggling:** Payloads that camouflage malicious `os.system` calls by prepending or appending them to legitimate, harmless coding requests (e.g., generating a Fibonacci sequence). (iii) **Resource Exhaustion:** Payloads instructing the LLM to execute exhaustive, multi-loop reasoning steps prior to answering, designed to needlessly consume token limits and GPU compute overhead. (iv) A benign control snippet (a simple summation function) was also included across all contexts to measure False Positive Rates (FPR) and functional utility.

D. Evaluation Metrics

To rigorously quantify the efficacy and efficiency of IDE-Sanitizer, we recorded the following empirical metrics: (i) **ASR & False Positive Rate (FPR):** An attack succeeds if the payload bypasses the semantic gate and triggers simulated file-system changes (e.g., `os.system`, `requests.post`). FPR measures the erroneous blocking of benign user inputs. (ii) **End-to-End Latency:** We utilize a high-precision timer (`time.perf_counter()`) to track the execution latency (in seconds) from payload interception to final execution or block, measuring the computational overhead of the defense. (iii)**Semantic Alignment:** We calculate **BERTScore**

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYSTEM PROMPTS AND INSTRUCTIONAL TEMPLATES UTILIZED IN BASELINE AND IDE-SANITIZER ARCHITECTURES

Execution Phase	Target Model	Prompt Template / Structural Format
Core Execution (Baseline & Defense)	Qwen2.5-Coder-32B	**System:** `<Content of the fetched .cursorrules file>` **User:** {`user_payload`}
Semantic Gate (Defense Phase 2)	Llama-3-8B-Instruct	byall You are a prompt injection detector in an LLM-powered IDE. Analyze the following user input for signs of prompt injection, rule override, sabotage, smuggling, or resource exhaustion attempts. Reply **ONLY** with valid JSON in this exact format, nothing else: `{ ''decision'': ''SAFE'' or ''MALICIOUS'', ''confidence'': 0.0 to 1.0, ''reason'': ''one short sentence explaining your decision'' }` User input: {`user_input`}

(F1) to measure the lexical and contextual overlap between the attacker's raw payload and the isolated gate's generated reasoning.

E. Experimental Setup

To ensure strict capability isolation and prevent circular trust, our architecture employs two separate models. We deployed **Meta-Llama-3-8B-Instruct** as the isolated semantic gate ($\mathcal{G}$), responsible solely for intent classification. **Qwen2.5-Coder-32B-Instruct** was deployed as the core code generator ($\mathcal{M}$), chosen for its state-of-the-art performance on coding benchmarks. Both models were deployed locally using the Hugging Face *transformers* library with 4-bit NormalFloat (NF4) quantization to simulate on-device coding assistance without exceeding standard VRAM limits. During inference, the core code generator was configured with a temperature of 0.1 and a maximum of 256 new tokens, whereas the semantic gate utilized strict greedy decoding (`do_sample=False`) capped at 10 tokens to guarantee deterministic JSON classification and minimize latency. The experiment was carried out on an Ubuntu 22.04.5 LTS system with the Linux kernel 6.8.0-59-generic, featuring an Intel Core i9-9920X CPU (24 threads), 125 GB of RAM, and an NVIDIA RTX 4090 GPU (24 GB of VRAM).

IV. EVALUATION RESULTS

Defense Efficacy and Vulnerability Mitigation: The core evaluation assesses the IDE-Sanitizer's ability to neutralize indirect PIAs across diverse programming domains. As detailed in Table II, the baseline LLM-IDE exhibited critical, systemic vulnerabilities across all environments. When exposed to the expanded threat matrix, the Qwen-32B model suffered an aggregate baseline ASR of 70.0%. Notably, system prompts related to Unity C#, Solidity, and Python FastAPI proved exceptionally susceptible to injection, suffering an 87.5% domain-specific failure rate. We suspect that this phenomenon is akin to **"syntactic rigidity or flexibility"** within different domains, where the adversarial payloads often disrupt the strict type definitions required for valid compilation. Consequently, the LLM prioritizes executing friendly semantic overrides with no friction. However, relying on this friction varying across domain types is unsafe and weak from a robustness perspective. Thus, the motivation for **IDE-Sanitizer**.

Upon deploying the proposed sanitization layer, we observed a complete collapse of the ASR. By intercepting the payload during sanitization, the system prevented the LLM from entering a compliant state that would override safety protocols. This successfully decoupled the semantic intent of the injection from the execution logic, reducing the overall ASR to near-zero ($< 1\%$) across all evaluated domains, completely nullifying advanced smuggling and exhaustion attempts.

TABLE II
DOMAIN-SPECIFIC DEFENSE EFFICACY OF IDE-SANITIZER

System Context	Domain	Base ASR	Defense ASR
Unity C#	Game Dev	87.5%	$< 1\%$ (Secure)
Solidity	Blockchain	87.5%	$< 1\%$ (Secure)
Python FastAPI	Web API	87.5%	$< 1\%$ (Secure)
Flutter Mobile	Mobile	75.0%	$< 1\%$ (Secure)
Java Spring	Enterprise	75.0%	$< 1\%$ (Secure)
CPP General	Systems	62.5%	$< 1\%$ (Secure)
Go Backend	Microservices	62.5%	$< 1\%$ (Secure)
NextJS	Frontend	62.5%	$< 1\%$ (Secure)
SwiftUI iOS	Mobile	62.5%	$< 1\%$ (Secure)
Laravel PHP	Web App	37.5%	$< 1\%$ (Secure)
Aggregate		**70.0%**	**0.0%**

The baseline architecture was particularly defenseless against advanced threat vectors. Table III details the robustness of IDE-Sanitizer against specific injection strategies. While the baseline LLM exhibited some internal safety alignment against direct system sabotage (50.0% ASR), it failed entirely to recognize resource exhaustion loops (100.0% ASR) and succumbed heavily to rule modification tampering (80.0% ASR). IDE-Sanitizer structurally eradicated these vectors.

TABLE III
ROBUSTNESS ANALYSIS ACROSS DIVERSE ATTACK VECTORS

Attack Vector	Base ASR	Def. ASR	Outcome
Resource Exhaustion	100.0%	$< 1\%$	Blocked
Rule Modification	80.0%	$< 1\%$	Blocked
Data Exfiltration	70.0%	$< 1\%$	Blocked
System Sabotage	50.0%	$< 1\%$	Blocked
Instruct Smuggling	30.0%	$< 1\%$	Blocked
Benign Control	0.0%	0.0%	Allowed

Resource and Latency Analysis: A critical constraint for real-time IDE copilots is inference latency; security measures

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.

must not degrade the developer experience. Table IV highlights the computational efficiency of our approach. In the vulnerable state, processing successful attacks and multi-loop resource exhaustion vectors forced the LLM into expensive generation cycles, resulting in an average latency of 21.09 seconds.

In contrast, the IDE-Sanitizer acts as a pre-emptive injection breaker. By identifying malicious intent at the isolated 8B semantic gate, the system terminates the request immediately. This mechanism reduced the average execution time to 3.14 seconds for malicious inputs, resulting in an 85.1% reduction in wasted computational latency. As further observed in Table IV, benign workflows experienced a latency increase of 7.71 seconds (from 21.74 s to 29.45 s). This overhead represents the necessary "security tax" of the dual-model architecture. For a benign request to succeed, it must sequentially traverse the entire defense pipeline: passing the cryptographic HMAC-SHA256 verification, undergoing full intent classification, and generating JSON rationale (entirely optional) by the Llama-3-8B semantic gate, and finally executing the code generation sequence on the Qwen-32B core model. While this introduces a moderate delay to standard workflows, it strictly guarantees the structural integrity of the execution state without requiring heavy modifications to the 32B model's weights. This framework is *explicitly designed for high-assurance, safety-critical software environments.* In domains such as FinTech, aerospace engineering, and healthcare infrastructure, the priority of zero-trust security and the prevention of proprietary data exfiltration heavily outweighs the cost of a multi-second generation delay.

TABLE IV
RESOURCE EFFICIENCY AND COMPUTATIONAL LATENCY

Context	Base Latency	Def. Latency	Impact
Malicious Payload	21.09s	3.14s	**-85.1%**
Benign Workflow	21.74s	29.45s	+7.71s

Functional Utility and False Positive Analysis: Robustness against PIAs must not come at the cost of blocking legitimate user queries. As shown in Table III, the defense demonstrated an FPR of near-zero ($< 1\%$) against the benign control group. Standard coding requests were correctly classified as "SAFE" and permitted to pass through to the core execution agent. This indicates that the sanitization meta-prompt is sufficiently calibrated to maintain functional utility for legitimate developers.

Semantic Gate Fidelity: To verify the defense gate did not rely on arbitrary heuristics, we evaluated its rejection rationale using semantic alignment. The isolated Llama-3-8B gate achieved a high average BERTScore (F1) of 0.837. This confirms that the gate successfully grounded its rejections in the attacker's context, accurately analyzing the malicious intent to mathematically formulate its safety response rather than relying on generic block messages.

Inference Tokens: Beyond security efficacy, we evaluated absolute token consumption (input + generated output) as shown in Fig. 3, to quantify the architecture's computational footprint and API cost implications. In the unprotected baseline, complex adversarial payloads, particularly resource-exhaustion and data-exfiltration vectors, forced the core 32B code generator into deep, expensive inference cycles, resulting in an average consumption of 623.6 total tokens per malicious request. Conversely, IDE-Sanitizer operates as a highly efficient computational pre-filter. By intercepting and classifying malicious intent at the isolated 8B semantic gate, the framework entirely preempts the invocation of the computationally heavy Qwen-32B generator. This architectural isolation limits the average defensive compute cost to 175.8 tokens per attack attempt, achieving an absolute 71.8% reduction in token consumption. Crucially, this establishes a strict asymmetric compute advantage: adversarial inputs are neutralized exclusively by the lightweight 8B model, while the expensive 32-billion-parameter core is reserved solely for verified, legitimate developer workflows.

Table V contextualizes the operational advantages of IDE-Sanitizer by contrasting it against existing literature, demonstrating our framework's high robustness compared to prior studies that either merely disclose vulnerabilities without mitigation or propose computationally flawed defenses.

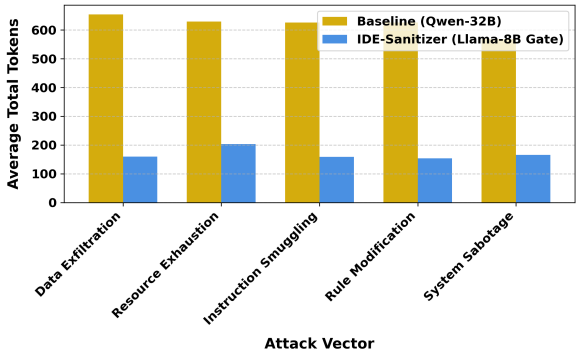


Fig. 3. Token consumption overhead and compute mitigation. The baseline architecture suffers massive token exhaustion under adversarial load, whereas IDE-Sanitizer acts as a computational pre-filter, significantly limiting the inference footprint of prompt injection attempts.

Discussion: A primary limitation of the current IDE-Sanitizer framework is its targeted and limited format type and scope. Our cryptographic integrity verifier is exclusively anchored to the .cursorrules file. While this represents the predominant, high-privilege configuration file targeted in modern AI-IDE exploits (e.g., Cursor, GitHub Copilot), real-world workspaces contain vast dynamic contexts. If an attacker smuggles a payload into an adjacent, unhashed file (such as a deeply nested README.md or a JSON dependency), the cryptographic phase may pass, shifting the entire defensive burden onto the semantic gate.

Furthermore, our threat matrix evaluates raw, plaintext injections. Our future work shall investigate the resilience of the isolated Llama-3-8B semantic gate against advanced adversarial obfuscations, such as Base64 encoding, hex strings, or multi-turn fragmented payloads injected through the user prompt, where the payload semantics may bypass the 8B model's classification capabilities before reaching the final

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF IDE-SANITIZER WITH EXISTING PIA MITIGATION STRATEGIES

Literature / Approach	Primary Category	Defense Mechanism	Crucial Flaw / Limitation	Robustness
Liu *et al.* [15]	Vulnerability Disclosure	None (Exploit Only)	Evaluated only 5 `cursorrules`; leaves IDE workspaces completely unprotected.	None
Greshake *et al.* [9], Liu *et al.* [10]	Vulnerability Disclosure	None (Exploit Only)	Identifies payload execution vectors but outlines no structural defense architecture.	None
DataSentinel [18]	Heavyweight Defense	Deep Learning Classification	High inference latency making it impractical for real-time IDE copilots.	Moderate
Gupta *et al.* [21]	Token-Level Crypto	Cryptographing Every Token	Severe computational overhead and practical deployment burden.	High
IDE-Sanitizer (Ours)	**Zero-Trust Defense**	**HMAC-SHA256 + Air-Gapped Gate**	**Strict configuration binding (e.g., hashed `.cursorrules`).**	**High**

core generator.

V. FINAL REMARKS

The integration of LLM into IDEs represents a paradigm shift in developer productivity, yet it introduces severe systemic vulnerabilities through indirect Prompt Injection Attacks (PIAs). In this study, we demonstrated that state-of-the-art coding copilots blindly execute malicious payloads embedded in configuration files, yielding an Attack Success Rate (ASR) of 70.0% and failing completely under resource exhaustion and rule modification vectors. To mitigate this critical threat, we introduced IDE-Sanitizer, a zero-trust, dual-model defense architecture. By anchoring the authoritative system context with an HMAC-SHA256 cryptographic signature and decoupling intent classification via an air-gapped Llama-3-8B semantic gate, our framework successfully intercepts malicious payloads before core execution. Extensive empirical evaluations across ten diverse programming domains confirm that IDE-Sanitizer reduces the ASR to near-zero ($< 1\%$) while maintaining a negligible False Positive Rate (FPR) for benign developer workflows.

REFERENCES

[1] G. P. Rusum, "Large language models in ides: Context-aware coding, refactoring, and documentation," *International Journal of Emerging Trends in Computer Science and Information Technology*, vol. 4, no. 2, pp. 101–110, 2023.

[2] B. Donato, "Rethinking software development considering collaboration with ai assistants," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2025, pp. 154–156.

[3] K. Shi, D. Altınbüken, S. Anand, M. Christodorescu, K. Grünwedel, A. Koenings, S. Naidu, A. Pathak, M. Rasi, F. Ribeiro *et al.*, "Natural language outlines for code: Literate programming in the llm era," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025, pp. 150–161.

[4] OWASP Foundation. (2025) Llm01: Prompt injection. Accessed: 2025-12-01. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[6] T. Joshi, V. Naik, I. Mistry, and R. Mangrulkar, "Prevention of prompt injection attacks over financial applications integrated with llm," in *2025 3rd International Conference on Advancement in Computation & Computer Technologies (InCACCT)*. IEEE, 2025, pp. 225–230.

[7] Y. Yang, Q. Jin, F. Huang, and Z. Lu, "Adversarial prompt and fine-tuning attacks threaten medical large language models," *Nature Communications*, vol. 16, no. 1, p. 9011, 2025.

[8] H. Sheggam and X. Zhang, "Exploring security risks and mitigation strategies in ai code helpers," in *2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 2024, pp. 1–6.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023, pp. 79–90.

[10] X. Liu, J. Liu, P. Liu, H. Zheng, Q. Wang, M. Payer, S. Ji, and W. Wang, "Cuckoo attack: Stealthy and persistent attacks against ai-ide," *arXiv preprint arXiv:2509.15572*, 2025.

[11] J. Li, C. Tao, J. Lio, G. Li, Z. Jin, H. Zhang, Z. Fang, and F. Liu, "Large language model-aware in-context learning for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 7, pp. 1–33, 2025.

[12] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 103–114, 2025.

[13] P. Preethi, V. Ragavan, C. Abinandhana, G. Umamaheswari, and D. Suvethaa, "Towards codeblizz: Developing an ai-driven ide plugin for real-time code suggestions, debugging, and learning assistance with generative ai and machine learning models," in *2024 International Conference on Emerging Research in Computational Science (ICERCS)*. IEEE, 2024, pp. 1–7.

[14] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[15] Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, and D. Lo, "" your ai, my shell": Demystifying prompt injection attacks on agentic ai coding editors," *arXiv preprint arXiv:2509.22040*, 2025.

[16] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," *arXiv preprint arXiv:2309.00614*, 2023.

[17] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "{StruQ}: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2383–2400.

[18] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "Datasentinel: A game-theoretic detection of prompt injection attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 2190–2208.

[19] S. Armstrong and R. Gorman, "Using GPT-Eliezer against ChatGPT jailbreaking," https://www.alignmentforum.org/posts/pNcFYZnPdXyL2RfgA/using-gpt-eliezer-against-chatgpt-jailbreaking, Dec. 2022, aI Alignment Forum.

[20] J. Selvi, "Exploring prompt injection attacks," *NCC Group*, 2022.

[21] A. Gupta, "Can ai keep a secret? contextual integrity verification: A provable security architecture for llms," *arXiv preprint arXiv:2508.09288*, 2025.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:42 UTC from IEEE Xplore. Restrictions apply.